

Collaborate Authentication Service – Design Document

Overview

The Collaborate Authentication Service is a RESTful API responsible for authenticating users through any Identity Provider (for this exercise a mock authorization service will be used), validating access tokens, and exposing the authenticated user's authorization information to downstream services. The service centralizes authentication and authorization logic, allowing other applications to rely on a single source of truth instead of implementing authorization independently.

The solution is built using **ASP.NET Core**, **Entity Framework Core**, and **OAuth 2.0 / OpenID Connect**, following a layered architecture to separate business logic, infrastructure, and API concerns.

Architecture

The application is organized into logical layers, each with a single responsibility.

- Application
- CollaborateAPI (Client)
- Domain
- Infrastructure
- Shared

The API layer exposes endpoints for authentication and authorization. Business logic resides in the application layer, while the infrastructure layer integrates with persistence layer. Entities resides in the Domain layer.

Authentication Flow

Identity Provider Abstraction

The exercise explicitly states that implementing a complete Identity Provider is out of scope. Therefore, the solution provides a single `MockAuthenticationProvider` implementing

`IAuthenticationProvider`. This component simulates the result of an external OIDC/SAML Identity Provider by returning a mocked `ExternalAuthenticationResult`.

In a production implementation, the `MockAuthenticationProvider` would be replaced by concrete implementations such as:

- `CasewareOidcAuthenticationProvider`
- `MicrosoftEntraAuthenticationProvider`
- `SamlAuthenticationProvider`

The `AuthenticationService` depends only on the `IAuthenticationProvider` abstraction, so no changes to the orchestration logic are required when introducing new providers.

Authorization

After successful authentication, the service extracts user claims from the validated token and resolves the user's permissions within the application.

Rather than requiring downstream services to understand authorization rules, the service exposes an endpoint that returns the current user's authorization context. This design centralizes authorization logic and ensures consistent permission evaluation across multiple services.

A typical response contains:

- User identifier
- Tenant identifier (when available)
- Roles
- Application permissions

This allows consuming services to authorize requests without querying the database or implementing their own permission logic.

Security Considerations

The solution follows common security practices:

- Authentication is delegated to Microsoft Entra ID.
- JWT signatures are validated before processing requests.
- Token issuer, audience, and lifetime are verified.

- Authorization decisions are based on validated claims.
 - HTTPS is required for all communication.
 - Sensitive secrets and configuration values are intended to be stored outside the source code (e.g., environment variables or Azure Key Vault).
-

Design Decisions

Several design decisions were made to improve maintainability and scalability:

- **Separation of concerns:** Authentication, authorization, persistence, and API responsibilities are isolated into different layers.
 - **Claims-based authorization:** The application relies on JWT claims instead of session state, making the API stateless.
 - **Dependency Injection:** Core services are registered through ASP.NET Core's built-in dependency injection container, improving testability.
 - **Entity Framework Core:** EF Core is used for data access, simplifying persistence while allowing future database providers to be introduced with minimal changes.
 - **MockAuthorizationProvider** acts as the **Identity Provider**, simulating the OAuth 2.0 authorization endpoint and issuing authorization codes.
 - **Authentication Service** acts as the **OAuth client and token issuer for your application**. It validates the authorization code and PKCE proof, creates your application's JWT, and establishes the authenticated identity.
 - **Authorization Service** consumes that authenticated identity and resolves application-specific permissions, roles, and tenant context.
-

Future Improvements

Although the current implementation satisfies the assignment requirements, several enhancements would improve production readiness:

- Implement refresh token support.
- Introduce role- and policy-based authorization using ASP.NET Core Authorization Policies.
- Cache authorization information to reduce database lookups.
- Add structured logging and distributed tracing.
- Expand automated unit and integration test coverage.
- Support multiple identity providers. **Support multiple Identity Providers:** The authentication architecture follows the **Open/Closed Principle (OCP)** by introducing an abstraction between the Authentication Service and the Identity Provider. The service depends on an interface rather than a concrete implementation,

allowing new Identity Providers (e.g., Microsoft Entra ID, Auth0, Keycloak, or other OAuth 2.0/OpenID Connect providers) to be added by implementing the abstraction without modifying the existing authentication logic. This approach improves extensibility, maintainability, and reduces coupling to any specific authentication vendor.

Conclusion

The Authentication and Authorization services act as orchestration layers between the application and the external Identity Provider. Rather than embedding provider-specific logic throughout the system, these services encapsulate identity validation, user resolution, and permission evaluation behind application-defined interfaces. This abstraction decouples the rest of the system from any Identity Provider, enabling downstream services to consume a consistent authorization model independent of the underlying identity provider. As a result, future migration to another OAuth 2.0/OpenID Connect provider (such as Okta, Auth0, or Keycloak) would primarily affect the infrastructure layer, minimizing changes to application and business logic.